

Great. Here's **Part 2 (Questions 21–40)** covering the **most commonly asked JavaScript interview questions**.

21. What is JavaScript?

Answer

JavaScript is a **single-threaded, synchronous, interpreted (JIT-compiled)** programming language used to build interactive web applications.

It supports:

- Object-Oriented Programming (OOP)
- Functional Programming
- Asynchronous Programming
- Event-driven Programming

Modern JavaScript engines like V8 use **Just-In-Time (JIT)** compilation for performance.

22. Explain the JavaScript Execution Context.

An **Execution Context** is the environment where JavaScript code is executed.

There are three types:

1. Global Execution Context
2. Function Execution Context
3. Eval Execution Context (rarely used)

Each context has:

- Variable Environment
- Lexical Environment
- `this`

Example:

JavaScript

```
let x = 10;

function test() {
  let y = 20;
}
```

```
test();
```

Execution:

Global Context

↓

test() Context

↓

Back to Global

23. What is Hoisting?

Hoisting is JavaScript's behavior of moving **declarations** to the top of their scope before execution.

Example:

JavaScript

```
console.log(a);
```

```
var a = 5;
```

Actually becomes:

JavaScript

```
var a;
```

```
console.log(a);
```

```
a = 5;
```

Output:

```
undefined
```

24. Difference between var, let and const

var	let	const
Function scoped	Block scoped	Block scoped
Can redeclare	Cannot	Cannot
Can reassign	Can	Cannot
Hoisted with <code>undefined</code>	Hoisted but in TDZ	Hoisted but in TDZ

Example:

JavaScript

```
{
  var a = 1;
  let b = 2;
}

console.log(a); // 1

console.log(b); // Error
```

25. What is the Temporal Dead Zone (TDZ)?

TDZ is the period between entering a scope and the variable's declaration where `let` and `const` exist but cannot be accessed.

JavaScript

```
console.log(a);

let a = 5;
```

Output:

ReferenceError

26. What is a Closure?

A closure is a function that **remembers variables from its outer scope**, even after the outer function has finished executing.

Example:

JavaScript

```
function counter() {  
  let count = 0;  
  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
const c = counter();  
  
console.log(c()); // 1  
console.log(c()); // 2
```

Uses:

- Private variables
- Event handlers
- Timers
- Memoization
- Currying

27. What is Lexical Scope?

Lexical scope means a function can access variables from the scope where it was **defined**, not where it is called.

JavaScript

```
let a = 10;  
  
function test() {  
  console.log(a);  
}  
  
test();
```

Output:

10

The function remembers its surrounding scope.

28. Explain the Event Loop.

JavaScript is single-threaded.

The Event Loop allows asynchronous operations without blocking the main thread.

Flow:

Call Stack

↓

Web APIs

↓

Callback Queue

↓

Event Loop

↓

Call Stack

When the call stack is empty, the Event Loop moves queued callbacks onto it.

29. What are Microtasks and Macrotasks?

Microtasks

- Promise callbacks
- `queueMicrotask()`
- `MutationObserver`

Macrotasks

- `setTimeout`
- `setInterval`
- DOM events
- `setImmediate` (Node.js)

Priority:

Call Stack

↓

Microtasks

↓

Macrotasks

Microtasks always run before macrotasks.

30. Predict the output

JavaScript

```
console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 0);

Promise.resolve().then(() => {
  console.log("Promise");
});

console.log("End");
```

Output:

```
Start
End
Promise
Timeout
```

Why?

- Synchronous code runs first.
 - Promise callback goes to the **Microtask Queue**.
 - `setTimeout` goes to the **Macrotask Queue**.
 - Microtasks are processed before macrotasks.
-

31. What is a Promise?

A Promise represents the eventual completion or failure of an asynchronous operation.

States:

Pending

↓

Fulfilled

or

Rejected

Example:

JavaScript

```
const promise = fetch("/users");
```

Promises help avoid callback hell and improve readability.

32. Promise.all()

Runs multiple promises in parallel.

JavaScript

```
await Promise.all([
  fetch("/users"),
  fetch("/posts"),
  fetch("/comments")
]);
```

Behavior:

- Resolves when **all** promises succeed.
- Rejects immediately if **any one** promise rejects.

Use it for independent tasks where all results are required.

33. Promise.allSettled()

Waits for every promise to finish, regardless of success or failure.

JavaScript

```
await Promise.allSettled([
  fetch("/a"),
  fetch("/b"),
  fetch("/c")
]);
```

Returns an array with each promise's status (`fulfilled` or `rejected`).

Useful when partial failures are acceptable.

34. Promise.race()

Returns the result of the **first promise that settles**, whether fulfilled or rejected.

JavaScript

```
Promise.race([
  fetch("/api"),
  timeoutPromise
]);
```

Common use case:

- Implementing request timeouts.
-

35. Promise.any()

Returns the **first fulfilled promise**.

Rejected promises are ignored unless **all** promises reject.

JavaScript

```
Promise.any([
  fetch(server1),
  fetch(server2),
  fetch(server3)
]);
```

Useful for fallback servers or redundant APIs.

36. Difference between async/await and Promises

`async/await` is syntax built on top of Promises.

Promise style:

JavaScript

```
fetch("/users")
  .then(res => res.json())
  .then(console.log);
```

Async/await:

JavaScript

```
const res = await fetch("/users");
const data = await res.json();
```

Advantages:

- Easier to read
- Looks synchronous
- Better error handling with `try/catch`

37. Does async make code asynchronous?

No.

An `async` function **always returns a Promise**, but code only becomes asynchronous when you `await` an asynchronous operation.

Example:

JavaScript

```
async function test() {
  console.log("A");
  console.log("B");
}

test();
```

Output:

A
B

Now with `await`:

JavaScript

```
async function test() {  
  console.log("A");  
  
  await fetch("/users");  
  
  console.log("B");  
}
```

`B` runs after the `fetch` Promise resolves.

38. Explain `this` in JavaScript

The value of `this` depends on how a function is called.

Example:

JavaScript

```
const obj = {  
  name: "John",  
  greet() {  
    console.log(this.name);  
  }  
};  
  
obj.greet();
```

Output:

John

Regular function:

JavaScript

```
function test() {  
  console.log(this);  
}
```

In browsers (non-strict mode):

```
window
```

Arrow functions do **not** have their own `this`; they inherit it from the surrounding scope.

39. What are Arrow Functions?

Arrow functions provide a shorter syntax and **lexically bind** `this`.

JavaScript

```
const add = (a, b) => a + b;
```

Differences from regular functions:

- No own `this`
 - Cannot be used as constructors (`new`)
 - No `arguments` object
 - Best for callbacks and methods that need lexical `this`
-

40. Explain Debounce vs Throttle

Debounce

Executes the function **after** the user stops triggering the event for a specified delay.

Example:

- Search box
- Auto-save
- Form validation

JavaScript

```
function debounce(fn, delay) {  
  let timer;  
  
  return (...args) => {  
    clearTimeout(timer);  
  
    timer = setTimeout(() => {  
      fn(...args);  
    }, delay);  
  };  
}
```

```
};  
}
```

Throttle

Executes the function at most **once** during a specified interval.

Example:

- Scroll events
- Resize events
- Mouse movement

JavaScript

```
function throttle(fn, delay) {  
  let waiting = false;  
  
  return (...args) => {  
    if (waiting) return;  
  
    fn(...args);  
  
    waiting = true;  
  
    setTimeout(() => {  
      waiting = false;  
    }, delay);  
  };  
}
```

Senior Interview Follow-up Questions

Be prepared to answer these after the basics:

1. Why is JavaScript single-threaded?
 - It simplifies programming and avoids many concurrency issues. The Event Loop provides asynchronous behavior.
2. Why are Promises preferred over callbacks?
 - They avoid callback hell, improve readability, support chaining, and provide centralized error handling.
3. When should you use `Promise.all()` ?

- When tasks are independent and you need **all** results before continuing. They execute concurrently.
4. When should you use `Promise.allSettled()` ?
 - When every task should complete, even if some fail (e.g., uploading multiple files).
 5. When should you use debounce instead of throttle?
 - Debounce for actions after user input stops (search, validation). Throttle for limiting high-frequency events (scroll, resize).

These 20 JavaScript questions are among the most frequently asked in senior frontend interviews. The next part will cover **Next.js (SSR, SSG, ISR, Server Components, App Router, Middleware, caching, authentication)**, another major focus area for Senior Front-End Developer roles.